



55_card_predictor.zip
Zip Archive



Lab_4_VibeCoding_hand...
PDF

i have 4 tasks to be done i will give prompts to each one by one

task 1-Fix the card rank comparison bug

When predicting whether the next card is higher or lower, face cards and tens behave inconsistently. In

game_engine.evaluate_guess(), the comparison uses rank_str (e.g., comparing string "10" against "9", or "K" against "Q") instead of comparing numeric ranks. Because strings are compared lexicographically in Python, "10" is



Show more ▾

Yep. I checked the actual repo files and the Lab 4 handout. Task 1 is specifically the deliberate lexicographical-comparison bug described in the README/handout.

Lab_4_VibeCoding_handout

For **Task 1**, only `game/game_engine.py` needs to change. `card.py` already receives and stores `numeric_rank`, so we should preserve that design rather than unnecessarily changing it.

What changes

Current buggy logic:

Python

Run

```
self.next_card.rank_str > self.current_card.rank_str
```

This compares strings, so Python can produce nonsense like:

```
"10" < "2"
```

```
"K" < "Q" # lexicographic comparison, not card ranking
```

We'll instead compare:

Python

Run

```
self.next_card.numeric_rank > self.current_card.numeric_rank
```

The rank values are supplied by the deck, so I'll also verify that the deck maps J=11, Q=12, K=13, A=14 before considering Task 1 complete.

Sources



Task 2: Implement a win streak multiplier

Currently, each correct guess only gives a flat +1 point.

Implement a streak tracker in `game_engine` that monitors consecutive correct predictions. Award bonus multipliers or escalating points for maintaining a streak (e.g., 2x points at 3 wins in a row, 3x at 5 wins), and reset the streak counter back to zero on an incorrect guess.

what changes should be made here

For **Task 2**, the cleanest change is to put the entire streak system inside `game_engine.py`. You don't need to modify `card.py` because the streak is game-state logic, not card logic.

Changes to make

1. Add a streak counter

In `GameEngine.__init__()`:

Python

Run

```
self.win_streak = 0
```

This tracks consecutive correct guesses.

2. Update evaluate_guess()

When the prediction is **correct**:

Python

Run

```
self.win_streak += 1
```

Then determine the multiplier:

Python

Run

```
if self.win_streak >= 5:
    multiplier = 3
elif self.win_streak >= 3:
    multiplier = 2
else:
    multiplier = 1
```

Then award:

Python

Run

```
self.score += multiplier
```

So the scoring becomes:

Consecutive wins	Multiplier	Points
1	1×	+1
2	1×	+1
3	2×	+2
4	2×	+2
5+	3×	+3

Example:

- Correct → streak 1 → +1
- Correct → streak 2 → +1
- Correct → streak 3 → +2
- Correct → streak 4 → +2
- Correct → streak 5 → +3

Wrong → streak 0 → +0

Correct → streak 1 → +1

3. Reset the streak on an incorrect guess

Inside the incorrect branch:

Python

Run

```
self.win_streak = 0
```

So a wrong prediction immediately breaks the streak.

One important design choice

I'd recommend **not resetting the streak after a correct guess**, obviously, and **not adding a separate "bonus" on top of +1**.

For example, at streak 3:

Python

Run

```
score += 2
```

rather than:

Python

Run

```
score += 1
score += 2
```

Otherwise you're effectively giving 3 points at the 3-win threshold, which doesn't match the requested **2× multiplier**.

Expected structure

Conceptually, `evaluate_guess()` becomes:

Python

Run

```
def evaluate_guess(self, guess):
    # Compare current and next cards
    # ...
```

```

if correct:
    self.win_streak += 1

    if self.win_streak >= 5:
        multiplier = 3
    elif self.win_streak >= 3:
        multiplier = 2
    else:
        multiplier = 1

    points = 1 * multiplier
    self.score += points

    return True
else:
    self.win_streak = 0
    return False

```

I'd also have the method expose the **points earned / current streak** if the existing game UI needs to display something like 🔥 Streak: 5 | +3 points . But if the current code expects `evaluate_guess()` to return only `True/False` , **don't change its return type unnecessarily**—just maintain the streak internally.

So for your assignment, the key modifications are:

game_engine.py

- Add `self.win_streak = 0`
- Increment on correct prediction
- 3 consecutive wins → 2×
- 5+ consecutive wins → 3×
- Reset to 0 on wrong prediction
- Apply multiplier to the points awarded

card.py

- No change needed for Task 2.

This also keeps Task 2 nicely isolated from Task 1, which is exactly what you want when you're doing the four prompts one-by-one.



Task 3: Implement tie / push handling

When the drawn card has the exact same rank as the current card (e.g., 7 of Hearts followed by 7 of Spades), the guess is automatically penalized as incorrect. Implement custom tie/push rules: preserve the player's score and streak, and display a yellow "PUSH / TIE! Rank matched." message rather than deducting a point.
implement this

Yep. For Task 3, the important part is that a tie is **neither correct nor incorrect**. It is a **push**, so:

- Score stays exactly the same.
- Streak stays exactly the same.
- No multiplier is applied.
- Display: PUSH / TIE! Rank matched.
- Message should be **yellow**.
- Normal higher/lower logic should only run when the ranks are different.

Change game_engine.py

The evaluation order should be:

Python

Run

```
current_rank = self.current_card.numeric_rank
next_rank = self.next_card.numeric_rank

# Tie / Push
if next_rank == current_rank:
    # Do NOT change score
    # Do NOT change streak
    return "PUSH / TIE! Rank matched."

# Normal guess evaluation
if guess == "higher":
    correct = next_rank > current_rank
elif guess == "lower":
    correct = next_rank < current_rank
```

Then your Task 2 streak logic can follow the correct result:

Python

Run

```
if correct:
    self.win_streak += 1

    if self.win_streak >= 5:
        multiplier = 3
    elif self.win_streak >= 3:
        multiplier = 2
    else:
        multiplier = 1

    self.score += multiplier
else:
    self.win_streak = 0
```

Important: don't reset the streak on a tie

This is the key difference:

Higher/lower CORRECT

↓

streak +1
score increases

Higher/lower WRONG

↓

streak = 0
score decreases/normal penalty

SAME RANK / TIE

↓

streak unchanged
score unchanged
"PUSH / TIE! Rank matched."

For example:

Score = 7
Streak = 4

Current: 7♥
Next: 7♠
Guess: Higher

```
→ PUSH / TIE! Rank matched.  
→ Score = 7  
→ Streak = 4
```

Then if the next round is correct:

```
Streak = 5  
→ 3× multiplier  
→ +3 points
```

Yellow message

The **game UI/display code** needs to handle the returned push result and render it in yellow. Don't hard-code terminal color codes into `evaluate_guess()` if your project already has a UI layer.

If the existing project uses Tkinter, for example:

Python	Run
<pre>message_label.config(text="PUSH / TIE! Rank matched.", fg="yellow")</pre>	

If it uses a different UI mechanism, use that project's existing message/display function.

One thing I'd preserve

If `evaluate_guess()` currently returns a boolean (`True` / `False`), **don't blindly replace it with a string** because that could break the rest of the game.

A safer implementation is to introduce a distinct result such as:

Python	Run
<pre>"PUSH"</pre>	

or whatever result structure the existing code already uses, and let the UI translate that into:

```
PUSH / TIE! Rank matched.
```

So the core Task 3 change is: check `numeric_rank == numeric_rank` **before** evaluating higher/lower, and return a push result without touching `score` or `win_streak` .

If you send me the current `game_engine.py` (or let me inspect the uploaded ZIP), I can give you the **exact full updated file for Task 3**, without guessing how your existing return/UI logic works.



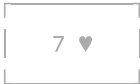
Task 4: Implement side-by-side card reveal animation

Currently, the current card immediately switches to the new card on guess submission. Modify `game_engine.render()` to display both cards side-by-side (the previous card on the left and the newly revealed card on the right) with a brief pause or transition before shifting to the next round, giving the player time to visually confirm both cards.
implement this

For **Task 4**, the main change should be in `game_engine.render()` .

The desired flow is:

Before `guess`:



Player `guesses` HIGHER

Reveal:



Previous

New

↓ brief pause

Next round:



What needs to change

You want to **preserve the old card before replacing** `current_card`.

Something along these lines:

Python

Run

```
previous_card = self.current_card
self.current_card = self.next_card
```

Then call `render()` with both cards available during the reveal.

The rendering logic should essentially be:

Python

Run

```
def render(self, previous_card=None, reveal=False):
    # Clear/update the game display

    if reveal and previous_card is not None:
        # Show previous card on LEFT
        self.render_card(previous_card, position="left")

        # Show newly revealed card on RIGHT
        self.render_card(self.current_card, position="right")

        # Brief pause so player can see both
        time.sleep(1)

    else:
        # Normal single-card display
        self.render_card(self.current_card, position="center")
```

But there's an important thing

Don't blindly use `time.sleep()` if your game has a GUI.

If this is a Tkinter-style application, `time.sleep(1)` freezes the entire GUI. In that case you should use something like:

Python

Run

```
self.root.after(1000, self.start_next_round)
```

That lets the UI remain responsive while the two cards are displayed.

If it's a terminal-based game, a simple:

Python

Run

```
time.sleep(1)
```

is perfectly fine.

Recommended game flow

The guess-processing code should become conceptually:

Python

Run

```
previous_card = self.current_card

# Draw/reveal the next card
self.current_card = self.next_card

# Show both cards
self.render(
    previous_card=previous_card,
    reveal=True
)

# After the reveal pause, continue to next round
self.next_card = self.draw_card()
self.render()
```

The important distinction is:

```
previous_card = card BEFORE guess
current_card  = newly revealed card
```

rather than immediately overwriting the old card and losing it.

If `render()` currently has no parameters

You can keep the existing API and temporarily store the previous card:

Python

Run

```
self.previous_card = self.current_card
self.current_card = self.next_card
self.render()
```

Then inside `render()`:

Python

Run

```
if self.previous_card is not None:
    # render previous_card on left
    # render current_card on right
else:
    # render current_card normally
```

After the reveal:

Python

Run

```
self.previous_card = None
```

so the next normal render goes back to displaying one card.

Task 4 requirements checklist

Your implementation should satisfy all four:

- ☒ Previous card displayed on the **left**
- ☒ Newly revealed card displayed on the **right**
- ☒ Both remain visible briefly before advancing
- ☒ Normal gameplay resumes with the newly revealed card as the current card

Since you're doing **Tasks 1–4 sequentially**, I'd make sure Task 4 doesn't accidentally undo the Task 2 streak or Task 3 push behavior. The architecture should be:

Guess

↓

`evaluate_guess()`

```
|— Tie → preserve score + streak
|— Correct → streak + multiplier
|— Wrong → reset streak
↓
Reveal cards
↓
render(previous_card, current_card)
↓
brief pause
↓
Next round
```

If you want the **exact full modified** `game_engine.py` **for all 4 tasks together**, I can work directly from your uploaded ZIP and give you the final file rather than having you manually merge these snippets.



ChatGPT can make mistakes. Check important info.